

# OTA远程升级 BootLoader

---

## 1 两段式BootLoader程序

### 1.1 系统组成与角色分工

#### 1.1.1 单片机内部Flash B区

#### 1.1.2 单片机内部Flash A区

#### 1.1.3 W25Q32

#### 1.1.4 W24C02

### 1.2 工作原理与流程（含状态机）

#### 1.2.1 状态标志位示例

#### 1.2.2 正常启动流程（无升级操作）

#### 1.2.3 OTA升级流程

### 1.3 为什么B区在前A区在后呢？

#### 1.3.1 硬件决定的启动流程（根本原因）

#### 1.3.2 工程实践的优化与可靠性

#### 1.3.3 简化应用程序（APP）的设计

#### 1.3.4 两种分区方案的对比

## 2 三段式BootLoader程序

### 2.1 核心设计理念

#### 2.1.1 Flash内存划分区

#### 2.1.2 关键状态机（存储在配置信息区）

### 2.2 系统启动与升级工作流程

#### 2.2.1 正常启动流程（无升级操作）

#### 2.2.2 固件升级（OTA）流程

#### 2.2.3 升级失败回滚流程

### 2.3 方案优势总结

### 2.4 代码

### 2.5 OTA相关问题

#### 2.5.1 OTA状态位什么时候改变为可升级？

#### 2.5.2 OTA升级时，最新版本的程序下载到哪里了？

2.5.3 OTA升级时，最新版本的程序文件如何下载？下载多少？

2.5.4 下载时这个变量需不需要保存？

2.5.5 发生OTA事件时，B区如何更新到A区域？

## 2.6 OTA升级中断电

2.6.1 两种存储器的分工与特性

2.6.2 针对性的断电保护策略

2.6.3 系统上电恢复流程

2.6.4 总结

3

# 1 两段式BootLoader程序

## 1.1 系统组成与角色分工

### 1.1.1 单片机内部Flash B区

1. 功能：BootLoader程序
2. 角色：系统的总指挥和操作员。它是一段小巧、稳定、可靠的代码，唯一目的就是管理启动流程和固件更新。
3. 特点：这段程序通常永远不变（除非BootLoader本身需要升级）。它最先启动，负责检查状态、决策流程、执行擦写操作。

### 1.1.2 单片机内部Flash A区

1. 功能：应用程序（APP）
2. 角色：设备的业务逻辑核心。实现产品所有正常功能的代码都在这里，比如连接服务器、采集传感器数据、控制外设等。
3. 特点：这是需要被更新的对象。新版本固件其实就是为了替换这个区域的程序。

### 1.1.3 W25Q32

1. 功能：SPI Flash
2. 角色：“固件包仓库”或“中转站”。

3. **作用：** 由于单片机的内部Flash在擦写时不能同时执行代码，且网络下载固件包速度较慢，需要一个容量的外部存储来**临时存放**从网络接收到的**完整的新固件二进制文件**。BootLoader会从这里读取数据，然后写入到A区。

## 1.1.4W24C02

1. **功能：** EEPROM
2. **角色：** 系统的“记事本”或“状态管理中心”。
3. **作用：** 用来存储关键的状态标志位和元数据（如升级状态、APP版本号、CRC校验值等）。它的特点是字节级擦写、数据掉电不丢失，非常适合频繁更新少量关键数据。

## 1.2 工作原理与流程（含状态机）

整个系统的运行依赖于一个清晰的状态机，这些状态就存储在W24C02中。

### 1.2.1 状态标志位示例

1. `OTA_FLAG`：升级标志
  - a. `0xFF`：需要升级
  - b. `0x00`：正常启动
  - c. `0xAA`：升级成功
  - d. `0x55`：升级失败
2. `APP_Version`：当前A区应用的版本号
3. `APP_CRC`：当前A区应用的CRC校验值

### 1.2.2 正常启动流程（无升级操作）

1. **上电启动：** 单片机首先运行B区的BootLoader。
2. **读取状态：** BootLoader读取W24C02中的 `OTA_FLAG`。
3. **决策：** 发现 `OTA_FLAG == 0x00`（正常启动）。
4. **验证APP：**（可选但推荐）计算A区程序的CRC，与W24C02中存储的 `APP_CRC` 进行比对，验证程序完整性。
5. **跳转：** 验证通过后，BootLoader设置单片机的中断向量表偏移量，然后跳转到A区的起始地址执行应用程序。

6. 运行：设备开始正常运作。

### 1.2.3 OTA升级流程

#### 第1步：应用程序（A区）收到升级指令

1. 设备在A区程序运行中，通过4G网络收到服务器发来的升级通知（包含新版本号、固件大小等信息）。
2. A区程序将新固件通过SPI接口分包下载并存储到W25Q32中。
3. 下载完成后，A区程序计算W25Q32中固件的CRC，与服务器给出的CRC进行比对，确保下载无误。
4. 校验通过后，A区程序在W24C02中将 `OTA_FLAG` 写为 `0xFF`（需要升级），同时可以保存新版本号等信息。
5. 最后，A区程序主动触发软件复位（或看门狗复位），系统重启。

#### 第2步：BootLoader（B区）执行升级任务

1. 单片机复位，再次首先运行B区的BootLoader。
2. BootLoader读取W24C02，发现 `OTA_FLAG == 0xFF`。
3. BootLoader知道需要升级，于是开始操作：
  - **擦除A区**：擦除单片机内部Flash的应用程序区（A区）。
  - **搬运数据**：从W25Q32中读取新固件数据，写入到A区的对应地址。这个过程是“读取-写入-校验”循环，直到整个固件搬运完成。
  - **最终校验**：计算刚写入的A区程序的CRC，与之前存储的CRC值比对，确保万无一失。
4. **更新状态**：
  - 如果成功，将W24C02中的 `OTA_FLAG` 改为 `0xAA`（升级成功），并更新 `APP_Version` 和 `APP_CRC`。
  - 如果失败（如校验错误），则将 `OTA_FLAG` 改为 `0x55`（升级失败）。BootLoader可以设计为失败后不再跳转，等待工程师调试。
5. **重启或跳转**：BootLoader再次触发复位，或者直接跳转到A区。重启后，BootLoader会读到 `0xAA` 状态，验证A区程序后，跳转到新的应用程序，升级完成。

## 1.3 为什么B区在前A区在后呢？

核心原因在于：单片机芯片本身无法从任意地址启动，它总是从固定的起始地址开始执行代码。

### 1.3.1 硬件决定的启动流程（根本原因）

1. **固定入口地址：** 当单片机复位或上电后，硬件逻辑会强制将程序计数器（PC）指向一个特定的内存地址。对于绝大多数基于ARM Cortex-M内核的MCU（如STM32, GD32），这个地址是 0x08000000。这意味着芯片毫无例外地会从这个地址取出第一条指令开始执行。
2. **中断向量表的存放：** 程序开始的地方存放的不是普通代码，而是一个称为中断向量表的结构。这个表的第一个条目是初始栈指针（MSP）的值，第二个条目就是复位向量（Reset Handler），即程序开始执行的地址。
3. **结论：** 任何希望在上电后首先运行的代码，必须存放在这个固定的起始地址（0x08000000）上。因此，作为“开机第一件事”的BootLoader，必须被放置在这个地址，也就是Flash的最前端。

### 1.3.2 工程实践的优化与可靠性

1. **BootLoader的稳定性与独立性：** BootLoader是系统更新的最后一道防线。即使应用程序（A区）更新失败或完全损坏，一个独立且稳定的BootLoader仍然可以驻留在最前方的安全位置，等待下一次升级指令或进行恢复。如果APP在前，BootLoader在后，一旦APP区发生严重错误（例如错误地擦写了后面的扇区），可能导致整个系统瘫痪，无法恢复。
2. **大小固定，易于管理：** BootLoader的功能通常很精简（初始化、通信、擦写Flash），其大小是固定且可预估的（例如16KB或32KB）。将它放在开头，并分配一个固定大小的空间，使得内存规划非常清晰。后面的所有空间都可以灵活地分配给应用程序。

### 1.3.3 简化应用程序（APP）的设计

**中断向量表处理：** 这是非常重要的一点。每个程序（无论是BootLoader还是APP）都有自己的中断向量表。

1. 当CPU运行BootLoader时，它使用位于 0x08000000的向量表。
2. 当需要跳转到APP时，BootLoader会重映射中断向量表到APP区的起始地址。对于APP来说，它的起始地址就是  $0x08000000 + \text{BootLoader\_Size}$ 。
3. **链接脚本配置：** 在开发APP时，需要在IDE（如Keil）中配置链接脚本（Linker Script），告诉编译器代码应该从哪个地址开始存放。如果BootLoader在前，APP只需要简单地设置为一个固定的偏移地址（如 0x08004000）即可，非常直观和简单。
4. **Flash扇区布局的优化：** 许多单片机的Flash扇区大小并不是均匀的。通常前面的扇区较小，后面的扇区较大GD32F103的Flash，前4个扇区可能是16KB，后面的可能是64KB或128KB。
5. 小巧的BootLoader可以完美地放入一个或两个小扇区中，避免浪费空间。

6. 庞大的应用程序则可以放入后面的大扇区中，进行大块数据的存储和擦写，效率更高。

### 1.3.4 两种分区方案的对比

假设Flash总大小为256KB，BootLoader为32KB，APP为224KB。

#### (1) B区在前（正确且通用的方案）

B区 (BootLoader): 0x08000000– 0x08007FFF(32KB)

A区 (APP): 0x08008000– 0x0803FFFF(224KB)

工作流程：

1. 芯片从 0x08000000启动，运行BootLoader。
2. BootLoader检查是否需要升级。如果需要，则执行升级流程。
3. 如果不需要升级，BootLoader执行跳转：jump\_to\_app(0x08008000)。
4. APP代码开始运行，其链接地址就是 0x08008000，它首先会执行 SCB->VTOR = 0x08008000来重映射自己的中断向量表。

#### (2) A区在前（错误且麻烦的方案）

A区 (APP): 0x08000000– 0x08037FFF(224KB)

B区 (BootLoader): 0x08038000– 0x0803FFFF(32KB)

致命问题：

1. 芯片上电后，从 0x08000000启动，试图运行APP。
2. 但如果APP需要更新、损坏了、或者甚至不存在，设备就直接“变砖”了，因为根本无法运行到后面的BootLoader。
3. 要让BootLoader工作，你必须先通过仿真器烧录一个特殊的APP，这个APP的唯一功能是跳转到 0x08038000去执行BootLoader。这失去了BootLoader独立存在的意义，增加了不必要的复杂性和风险。

#### (3) 总结

将BootLoader（B区）放在单片机Flash的前部，而将应用程序（A区）放在后面，是由硬件启动机制决定的强制要求，同时也是为了实现系统的最大可靠性和工程上的简便性。这样做确保了：

1. 芯片总能启动：从固定地址总能找到可靠的BootLoader。
2. 更新安全：BootLoader作为一个独立的、不会被覆盖的“安全岛”，是系统能从错误中恢复的关键。
3. 开发简单：内存映射清晰，应用程序的链接配置简单明了。

## 2 三段式BootLoader程序

### 2.1 核心理念

BootLoader采用经典且可靠的A/B双备份系统（或称双系统），并配合独立的**配置信息区**，共同构成“三段式”结构。其核心目标是：**确保设备在任何升级状态下都不会“变砖”**，即至少有一个版本的应用程序（APP）可以正常工作。

#### 2.1.1 Flash内存划分区域

| 分区名称        | 功能描述                                              |
|-------------|---------------------------------------------------|
| Bootloader区 | 存放引导程序本身。负责系统启动决策、固件更新操作。此区域代码通常固定不变，是系统可靠性的基石。   |
| APP1区       | 应用程序备份一（通常作为工厂版本或稳定版本）。                           |
| APP2区       | 应用程序备份二（通常作为OTA更新版本）。                             |
| 配置信息区       | 存储关键系统状态标志和元数据（如升级状态、当前运行版本、CRC校验值等）。该区域数据需掉电不丢失。 |

#### 2.1.2 关键状态机（存储在配置信息区）

系统通过读取配置信息区中的状态标志来决定启动流程，这是一个简单的状态机，可能包含以下状态：

- 1. 升级待定（UPGRADE\_PENDING）：表示有新固件待安装。
- 2. 升级成功（UPGRADE\_SUCCESS）：表示新固件已成功验证并就绪。
- 3. 升级错误（UPGRADE\_ERROR）：表示上次升级过程失败。
- 4. 正常运行（APP\_VALID）：表示当前系统运行正常，无需升级操作。
- 5. 当前版本号、CRC校验值等。

## 2.2 系统启动与升级工作流程

### 2.2.1 正常启动流程（无升级操作）

1. **上电启动**：硬件强制从Flash起始地址开始运行，即执行BootLoader。
2. **读取状态**：BootLoader读取配置信息区中的状态标志（如 `OTA_FLAG`）。
3. **决策与验证**：若状态为“正常运行”或“升级成功”，BootLoader会进一步验证目标APP区（A区或B区）程序的完整性（如计算CRC并与存储值比对）。
4. **跳转执行**：验证通过后，BootLoader通过设置堆栈指针（SP）和程序计数器（PC），跳转到对应的APP区（A区或B区）执行应用程序。
5. **应用程序运行**：设备开始执行正常的业务逻辑功能。

### 2.2.2 固件升级（OTA）流程

#### 第1步：应用程序（APP）端准备

1. 设备在运行当前APP时（例如在APP1区），通过网络（如4G）接收到服务器下发的新固件包。
2. 当前APP将接收到的固件包临时存储到外部Flash（如文档中提到的W25Q32）中。
3. 下载完成后，APP计算临时固件的CRC，与服务器给出的校验值比对，确保下载无误。
4. 验证通过后，当前APP修改配置信息区中的状态标志，将其设置为“升级待定”（`UPGRADE_PENDING`），并写入新版本号等信息。
5. 最后，当前APP触发软件复位，系统重启。

#### 第2步：BootLoader端执行升级

1. 系统重启，再次进入BootLoader。
2. BootLoader读取状态，发现标志为“升级待定”（`UPGRADE_PENDING`）。
3. BootLoader开始执行升级操作：
  - a. **擦除目标区**：擦除将要被更新的APP区（例如APP2区）。
  - b. **固件搬运**：从外部Flash（W25Q32）中读取新固件数据，写入到目标APP区。
  - c. **最终校验**：计算刚写入的APP程序的CRC，与之前存储的校验值再次比对。
4. **更新状态**：
  - a. **成功**：则将状态修改为“升级成功”（`UPGRADE_SUCCESS`），并更新版本号 and CRC值。
  - b. **失败**：则将状态修改为“升级错误”（`UPGRADE_ERROR`）。BootLoader可设计为在此状态下不自动跳转，等待外部干预（如重新升级），从而避免启动一个不完整的程序。
5. **重启或跳转**：BootLoader再次触发复位，或者直接跳转到更新后的APP区。系统启动后，将运行新



版本的应用程序。

### 2.2.3 升级失败回滚流程

这是该设计**最关键的优势**所在。如果升级后验证失败（状态为“升级错误”），或者新程序本身无法运行：

1. BootLoader在启动时检测到错误状态。
2. 不会跳转到出问题的APP区（例如APP2）。
3. 而是自动回滚（Rollback）到已知的、稳定可用的另一个APP区（例如APP1，即工厂版本）运行。
4. 同时，BootLoader可以通过通信接口上报**错误日志**，告知上位机“升级失败，已回退至工厂版本”。

## 2.3 方案优势总结

1. **高可靠性（永不变砖）**：双备份机制确保即使升级失败，也有备份系统可启动，极大提高了系统可靠性。
2. **更新过程透明**：用户无感知，升级失败自动回退，不影响设备基本功能。
3. **标准化流程**：定义了清晰的状态机和升级流程，便于开发和维护。
4. **便于批量管理**：上位机可以方便地通过中控对网络中的所有传感器进行批量固件升级，非常适合工业场景。
5. **独立性强**：BootLoader、APP1、APP2相互独立，BootLoader作为最小核心系统，最为稳定。

此三段式BootLoader设计是该工业互联网项目稳定运行的基石，完美解决了远程设备维护和功能更新的核心痛点。

## 2.4 代码

### 2.4.1 擦除、写和读取内部Flash

1. Inf\_InteriorFlash.h

```
1  #ifndef __INF_INTERIORFLASH_H__
2  #define __INF_INTERIORFLASH_H__
3
4  #define EEPROM_START_ADDR ((uint32_t)0x0800F000)
5
6  #include "main.h"
7
8
9  /**
10   * @brief  擦除EEPROM区域的Flash页
11   * @param  StartPageAddress: 起始页地址 (必须是页对齐的地址)
12   * @param  NbPages: 要擦除的页数
13   * @retval HAL状态: HAL_OK成功, 其他值表示失败
14   */
15  HAL_StatusTypeDef FLASH_EraseEEPROM(uint32_t StartPageAddress, uint32_t Nb
    Pages);
16
17  /**
18   * @brief  向EEPROM区域写入数据
19   * @param  Address: 写入起始地址 (必须是半字对齐的地址)
20   * @param  Data: 指向要写入数据的指针
21   * @param  DataSize: 要写入的数据大小(单位: 半字)
22   * @retval HAL状态: HAL_OK成功, 其他值表示失败
23   */
24  HAL_StatusTypeDef FLASH_WriteEEPROM(uint32_t Address, uint16_t *Data, uint
    32_t DataSize);
25
26  /**
27   * @brief  从EEPROM区域读取数据
28   * @param  Address: 读取起始地址 (必须是半字对齐的地址)
29   * @param  Data: 指向存储读取数据的缓冲区指针
30   * @param  DataSize: 要读取的数据大小(单位: 半字)
31   */
32  void FLASH_ReadEEPROM(uint32_t Address, uint16_t *Data, uint32_t DataSize
    );
33
34
35  #endif
```

## 2. Inf\_InteriorFlash.c

```
1  #include "Inf_InteriorFlash.h"
2  #include "stm32f1xx_hal_flash.h"
3  #include "stm32f1xx_hal_flash_ex.h"
4  #include <stdio.h>
5
6  /* 全局变量，用于擦除配置 */
7  FLASH_EraseInitTypeDef EraseInitStruct;
8
9
10 /**
11  * @brief 擦除STM32F103内部FLASH的指定页
12  * @param StartPageAddress: 起始页地址（例如：0x0800F000），必须是页的起始地
    址
13  * @param NbPages: 要擦除的页数
14  * @retval HAL_StatusTypeDef: 操作状态 (HAL_OK, HAL_ERROR 或 HAL_BUSY)
15  */
16 HAL_StatusTypeDef FLASH_EraseEEPROM(uint32_t StartPageAddress, uint32_t NbPages)
17 {
18     HAL_StatusTypeDef status;
19     uint32_t PageError = 0;
20
21     /* 1. 解锁FLASH控制寄存器 */
22     HAL_FLASH_Unlock();
23
24     /* 2. 配置擦除参数 */
25     EraseInitStruct.TypeErase = FLASH_TYPEERASE_PAGES;
26     EraseInitStruct.PageAddress = StartPageAddress;
27     EraseInitStruct.NbPages = NbPages;
28
29     /* 3. 执行页擦除操作，并等待完成 */
30     status = HAL_FLASHEx_Erase(&EraseInitStruct, &PageError);
31
32     /* 4. 无论操作结果如何，都重新上锁FLASH以保护数据 */
33     HAL_FLASH_Lock();
34
35     /* 5. 可选：根据状态进行打印或处理 */
36     if (status == HAL_OK) {
37         printf("FLASH擦除成功!\r\n");
38     } else {
39         printf("FLASH擦除失败!错误页地址: 0x%08lX\r\n", PageError);
40     }
41
42     return status;
43 }
```

```

44
45 /**
46  * @brief 将数据写入STM32F103内部FLASH的指定地址
47  * @param Address: 写入起始地址（必须为偶数地址，半字对齐）
48  * @param Data: 指向要写入的数据数组的指针（16位半字数组）
49  * @param DataSize: 要写入的半字数据数量
50  * @retval HAL_StatusTypeDef: 操作状态（HAL_OK, HAL_ERROR 或 HAL_BUSY）
51  */
52 HAL_StatusTypeDef FLASH_WriteEEPROM(uint32_t Address, uint16_t *Data, uint32_t DataSize)
53 {
54     HAL_StatusTypeDef status = HAL_ERROR;
55     uint32_t i;
56
57     /* 1. 解锁FLASH控制寄存器 */
58     HAL_FLASH_Unlock();
59
60     /* 2. 循环写入每一个半字数据 */
61     for (i = 0; i < DataSize; i++) {
62         status = HAL_FLASH_Program(FLASH_TYPEPROGRAM_HALFWORD, Address +
63 (i * 2), Data[i]);
64         if (status != HAL_OK) {
65             /* 如果某次写入失败，立即跳出循环 */
66             break;
67         }
68     }
69
70     /* 3. 无论操作结果如何，都重新上锁FLASH */
71     HAL_FLASH_Lock();
72
73     /* 4. 可选：根据状态进行打印或处理 */
74     if (status == HAL_OK) {
75         printf("FLASH写入成功!写入%lu个半字.\r\n", DataSize);
76     } else {
77         printf("FLASH写入失败!在地址0x%08lx处出错.\r\n", Address + (i * 2));
78     }
79
80     return status;
81 }
82
83 /**
84  * @brief 从STM32F103内部FLASH的指定地址读取数据
85  * @param Address: 读取起始地址
86  * @param Data: 指向存储读取数据的缓冲区的指针
87  * @param DataSize: 要读取的半字数据数量
88  * @retval 无
89  * @note 此函数直接读取内存，通常不会失败。
90  */

```

```

90 void FLASH_ReadEEPROM(uint32_t Address, uint16_t *Data, uint32_t DataSize
91 )
92 {
93     uint32_t i;
94
95     /* 直接通过指针访问FLASH地址进行读取 */
96     for (i = 0; i < DataSize; i++) {
97         Data[i] = *(__IO uint16_t*)(Address + (i * 2));
98     }
99
100     printf("FLASH读取完成,从地址0x%08lX读取%lu个半字.\r\n", Address, DataSize
101 );
102 }

```

### 3. main.c

```

1  HAL_StatusTypeDef status;
2  uint16_t dataToWrite[4] = {0x1234, 0x5678, 0x9ABC, 0xDEFO}; // 要写入的数据
3  uint16_t dataRead[4] = {0}; // 用于存储读取数据的数组
4
5  /* 1. 擦除一页FLASH (STM32F103C8T6页大小为1KB) */
6  status = FLASH_EraseEEPROM(EEPROM_START_ADDR, 1);
7  if (status != HAL_OK)
8  {
9      Error_Handler(); // 处理错误
10 }
11
12 /* 2. 将数据写入已擦除的页 */
13 status = FLASH_WriteEEPROM(EEPROM_START_ADDR, dataToWrite, 4);
14 if (status != HAL_OK)
15 {
16     Error_Handler();
17 }
18
19 /* 3. 读取刚写入的数据进行验证 */
20 FLASH_ReadEEPROM(EEPROM_START_ADDR, dataRead, 4);
21 /* 4. 打印读取到的数据 */
22 printf("FLASH读取数据: 0x%04X 0x%04X 0x%04X 0x%04X\n",
23         dataRead[0], dataRead[1], dataRead[2], dataRead[3]);

```

```

[15:07:18.811]收←◆STM32 OTA
FLASH擦除成功!
FLASH写入成功!写入4个半字.
FLASH读取完成,从地址0x0800F000读取4个半字.
FLASH读取数据: 0x1234 0x5678 0x9ABC 0xDEFO

```

## 2.5 OTA相关问题

### 2.5.1 OTA状态位什么时候改变为可升级？

在程序于A区下载完成，并且经过校验确认无误后，将状态标志位修改为“可升级”。系统在下次重启时检测到该标志位，便会执行更新操作。

### 2.5.2 OTA升级时，最新版本的程序下载到哪里了？

分最新版本的程序以分片方式下载到外部Flash芯片W25Q32的A区。该芯片的每一页大小为256字节。

### 2.5.3 OTA升级时，最新版本的程序文件如何下载？下载多少？

升级过程由服务器下发通知，告知设备需要下载的新版本程序的总大小（单位：字节）。设备随后根据服务器指示的数据量进行下载。

### 2.5.4 下载时这个变量需不需要保存？

需要保存。从服务器获取的程序总大小等关键变量需要保存到非易失性存储器24C02中，以确保设备重启后仍能记住这些信息。

### 2.5.5 发生OTA事件时，B区如何更新到A区域？

设备重启后，在Bootloader中，会根据保存在24C02中的程序大小信息，从存储程序的A区读取数据，然后写入到用于运行的B区，完成程序更新。

## 2.6 OTA升级中断电

我的开发板同时使用了W24C02（EEPROM）和W25Q32（SPI Flash）这两种存储模块，我们来重新分析OTA升级过程中断电的问题。这两种芯片的特性不同，在系统中扮演的角色也不同，因此需要一套协同工作的策略来确保升级的可靠性。

### 2.6.1 两种存储器的分工与特性

| 存储芯片 | 类型 | 主要用途 | 关键类型 | 断电风险焦点 |
|------|----|------|------|--------|
|------|----|------|------|--------|

|        |                         |                                     |                                              |                                           |
|--------|-------------------------|-------------------------------------|----------------------------------------------|-------------------------------------------|
| W25Q32 | SPI NOR Flash           | 存储新版本程序文件<br>(容量大, 4MB)             | 以页 (256字节) 或扇区 (4KB) 为单位擦写; 写入前需先擦除; 擦写速度相对慢 | 写入/擦除大块程序数据时中断, 导致程序数据不完整或损坏。             |
| W24C02 | I <sup>2</sup> C EEPROM | 记录升级状态标志、程序大小等关键元数据<br>(容量小, 256字节) | 支持字节级擦写; 写入速度快, 无需先擦除。                       | 记录升级进度的关键状态位或数据在断电前未成功更新, 导致上电后无法准确判断断电点。 |

## 2.6.2 针对性的断电保护策略

### 1. 善用W24C02作为“升级状态记录器”

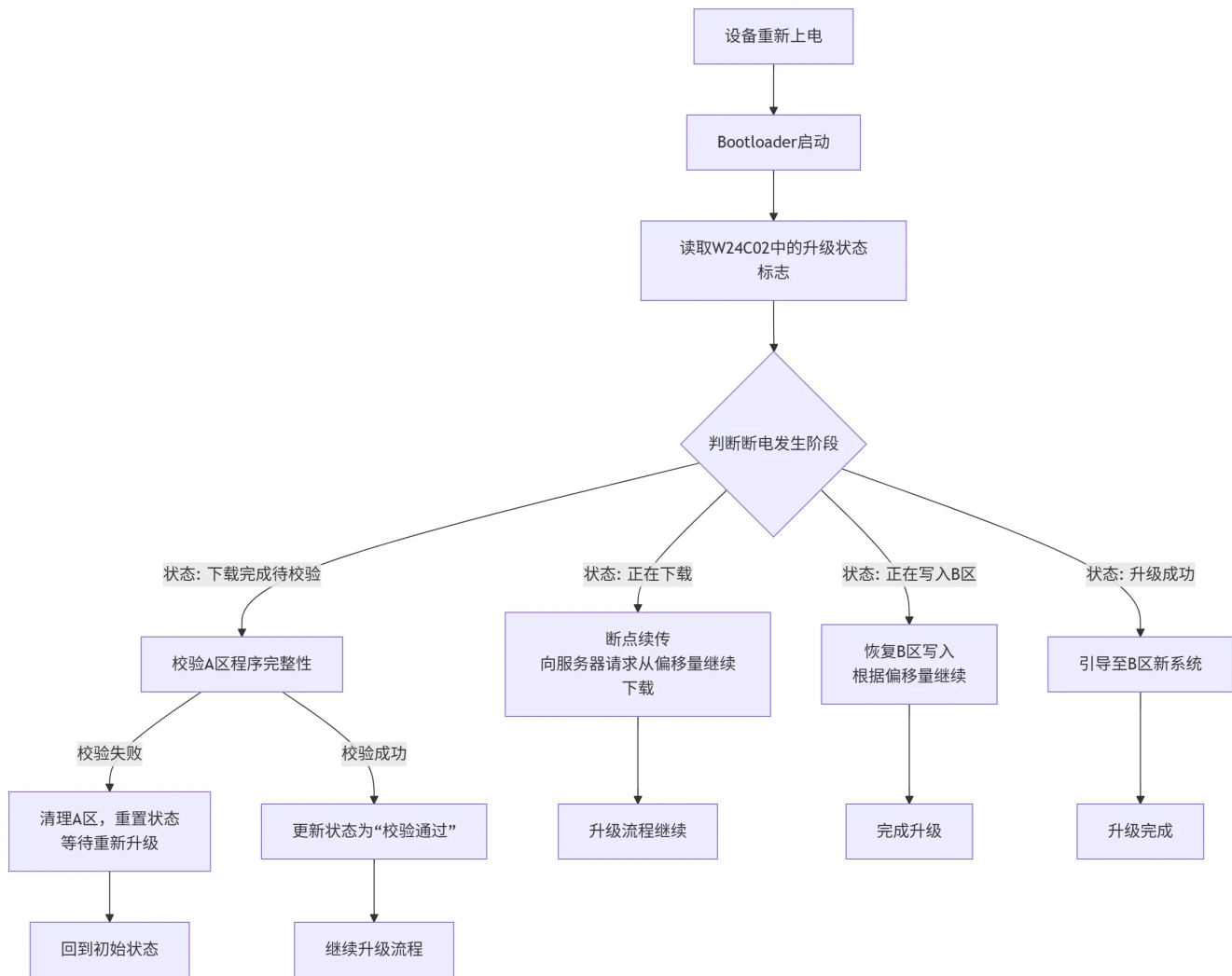
- 记录详细进度: 不仅记录“是否可升级”的最终标志, 更关键的是记录升级过程中的关键步骤状态。例如: `0x00`:等待升级、`0x01`:正在下载、`0x02`:下载完成待校验、`0x03`:校验通过待写入、`0x04`:正在写入B区、`0x05`:升级成功。
- 记录关键数据: 将服务器下发的程序总大小、当前已下载/已写入的偏移量等数据保存在W24C02中。
- 原子操作状态: 在W24C02上更改状态标志时, 尽量确保一次写入完成, 避免状态值处于中间的不确定状态。

### 2. 优化W25Q32的固件存储操作

- A/B分区与校验: 这是核心机制。新程序必须下载到非当前运行的分区 (例如A区)。下载完成后, 在重启前, 应对整个A区镜像进行CRC32或SHA256校验, 并将校验值也保存在W24C02中。只有当校验通过后, 才在W24C02中设置“校验通过”状态。
- 断点续传: 下载阶段, 利用W24C02保存已下载量。断电重启后, 可以向服务器请求从断点继续下载, 避免重复传输。
- 谨慎的写入顺序: 在将A区数据写入B区时, 可以考虑最后更新程序入口向量或校验和。这样, 即使写入过程中断电, Bootloader在检查时发现新固件不完整, 也不会贸然引导, 而是触发恢复流程。

## 2.6.3 系统上电恢复流程

设备重新上电后, Bootloader应执行以下决策流程, 其核心是读取W24C02中保存的状态信息, 以判断断电发生的具体阶段, 并执行相应的恢复操作:



### 2.6.4 总结

通过让 W24C02（EEPROM）和 W25Q32（SPI Flash）协同工作，一个负责精准记录“升级进度日记”，一个负责安全存储大体积的固件数据，并配合A/B分区、校验、断点续传等软件策略，可以构建一个相当健壮的OTA升级系统，能够有效应对升级过程中的意外断电情况。

## 3